

DATA ENGINEERING NA PRÁTICA

10 Boas Práticas de Segurança que Todo Iniciante Deve Conhecer

[Imagem de capa gerada por IA — ver prompts/prompt-capa.md]

Alexandre

Growth Data Engineer • Marketing Analytics

Introdução

A Engenharia de Dados é a disciplina responsável por projetar, construir e manter os sistemas que coletam, transformam e disponibilizam dados para análises, produtos e modelos de inteligência artificial. Enquanto o Cientista de Dados extrai insights, o Engenheiro de Dados garante que os dados cheguem até ele com qualidade, consistência e no tempo certo. Isso envolve pipelines de ETL/ELT, bancos de dados relacionais e não relacionais, data warehouses como BigQuery, orquestração de tarefas e, cada vez mais, integração com APIs de terceiros e serviços em nuvem.

Trabalhar com dados significa, na prática, trabalhar com informação sensível: credenciais de clientes, comportamento de usuários, dados financeiros e de negócio. Por isso, segurança não é um extra opcional — é parte do design do pipeline desde a primeira linha de código. Um vazamento de chave de API, uma injeção de SQL ou um log com dados pessoais expostos podem custar a reputação de uma empresa inteira. Todo profissional que pretende atuar na área, especialmente quem está começando, precisa incorporar práticas seguras como hábito, não como remendo depois que algo dá errado.

É aqui que a Inteligência Artificial generativa se torna uma aliada poderosa. Ferramentas como ChatGPT, Claude, Gemini e GitHub Copilot aceleram a escrita de scripts de ETL, sugerem tratamentos de erro, ajudam a revisar queries SQL em busca de vulnerabilidades e até geram documentação técnica. Usada com senso crítico, a IA não substitui o conhecimento do engenheiro — ela multiplica sua produtividade, liberando tempo para focar em arquitetura, qualidade e segurança dos dados. Este ebook reúne, de forma direta e prática, as boas práticas de segurança que todo iniciante em Engenharia de Dados deveria conhecer antes de colocar um pipeline em produção.

Arquitetura de um Pipeline Moderno

Um pipeline de dados moderno segue um fluxo bem definido, onde cada etapa tem uma responsabilidade clara. Entender essa arquitetura é o primeiro passo para saber onde e como aplicar segurança em cada camada.

API → Python → Validação → PostgreSQL → BigQuery → Dashboard

API — Origem dos dados

Os dados geralmente chegam por meio de uma API externa (CRM, plataforma de anúncios, ERP) ou de eventos gerados pela própria aplicação. Aqui já entra o primeiro cuidado: nunca expor chaves de API em código-fonte.

Python — Extração e transformação

Scripts em Python consomem a API, tratam o formato dos dados (JSON, CSV) e aplicam transformações. Bibliotecas como requests, pandas e psycopg2 são comuns nessa etapa.

Validação

Antes de qualquer gravação em banco, os dados passam por validação de schema, tipos e valores obrigatórios, evitando que dados corrompidos ou maliciosos avancem no pipeline.

PostgreSQL e BigQuery

O PostgreSQL costuma armazenar dados operacionais e transacionais, enquanto o BigQuery funciona como data warehouse analítico, otimizado para consultas em grande volume.

Dashboard

Por fim, os dados chegam a ferramentas como Power BI ou Looker Studio, onde geram valor de negócio através de indicadores e relatórios.

Boas Práticas de Segurança

A seguir, as práticas essenciais que devem estar presentes em qualquer pipeline de dados, independentemente do tamanho do projeto.

1. Variáveis de ambiente (.env)

Nunca escreva senhas, tokens ou strings de conexão diretamente no código. Use um arquivo .env para armazenar essas informações localmente e bibliotecas como python-dotenv para carregá-las em tempo de execução.

```
# .env
DB_HOST=localhost
DB_USER=admin
DB_PASSWORD=*****

# app.py
from dotenv import load_dotenv
import os
load_dotenv()
db_pass = os.getenv('DB_PASSWORD')
```

2. .gitignore

Garanta que arquivos sensíveis nunca sejam versionados. Um .gitignore bem configurado impede que .env, chaves .pem e credenciais de serviço subam para o GitHub.

3. Prevenção de SQL Injection

Sempre use queries parametrizadas em vez de concatenar strings. Isso impede que um valor malicioso vindo do usuário altere a lógica da consulta.

```
# Errado
cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")

# Correto
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

4. Logging estruturado

Registre eventos importantes do pipeline (início, fim, erros) usando a biblioteca logging, nunca print(). Evite logar dados sensíveis como senhas ou dados pessoais em texto puro.

5. Tratamento de erros

Use blocos try/except específicos para prever falhas de rede, timeout de API ou erro de banco, evitando que o pipeline pare de forma abrupta ou exponha stack traces sensíveis.

6. IAM (Identity and Access Management)

Na nuvem, cada serviço deve ter apenas as permissões estritamente necessárias (princípio do menor privilégio). Evite usar contas de administrador para tarefas automatizadas.

7. Gerenciamento de Secrets

Em produção, prefira serviços como Google Secret Manager, AWS Secrets Manager ou Docker Secrets em vez de arquivos .env, garantindo rotação e controle de acesso às credenciais.

8. Validação de entrada

Toda informação vinda de fontes externas deve ser validada quanto a tipo, formato e limites antes de ser processada ou persistida.

9. Criptografia em trânsito e em repouso

Utilize conexões SSL/TLS entre serviços e criptografia de dados sensíveis armazenados, especialmente em ambientes de nuvem.

10. Revisão de dependências

Mantenha bibliotecas atualizadas e monitore vulnerabilidades conhecidas com ferramentas como pip-audit ou Dependabot no GitHub.

Projeto Real: Secure Marketing Data Pipeline

Para tornar essas práticas concretas, apresento um projeto real aplicado ao meu contexto de Growth Data Engineering: um pipeline que coleta dados de marketing de APIs externas (como plataformas de anúncios e CRM), trata e valida essas informações, e as disponibiliza para análise em dashboards.

Arquitetura utilizada

- API: consumo de dados de campanhas via requisições autenticadas com token armazenado em variável de ambiente
- ETL: scripts em Python responsáveis por extração, limpeza e padronização dos dados
- PostgreSQL: camada operacional para armazenamento estruturado e auditável
- Cloud: BigQuery como data warehouse analítico, com IAM restringindo acesso por serviço
- Dashboard: visualização dos indicadores de performance de campanhas

Ao longo do desenvolvimento, apliquei validação de schema antes da gravação no banco, queries parametrizadas para evitar SQL Injection, logging estruturado para rastrear falhas de execução e um .gitignore configurado desde o primeiro commit. O resultado é um pipeline pequeno, mas construído com os mesmos princípios de segurança usados em ambientes corporativos — o que faz toda a diferença na hora de apresentar o projeto em uma entrevista técnica.

Como a IA Ajuda no Dia a Dia do Engenheiro de Dados

ChatGPT

Útil para gerar rascunhos de scripts ETL, explicar mensagens de erro e sugerir queries SQL, funcionando como um par de programação disponível 24 horas.

Claude

Especialmente forte em revisão de código extensa, refatoração e análise de segurança, ajudando a identificar más práticas antes de o código ir para produção.

Gemini

Integra-se bem ao ecossistema Google Cloud, sendo útil para tarefas envolvendo BigQuery, Looker Studio e demais serviços do GCP.

GitHub Copilot

Atua diretamente no editor de código, autocompletando funções e sugerindo testes, o que acelera especialmente tarefas repetitivas de ETL.

O ponto central é usar a IA como acelerador, sempre revisando criticamente o que é gerado — sobretudo em pontos sensíveis como segurança, tratamento de credenciais e lógica de negócio.

Checklist Profissional

- Código organizado em módulos e funções reutilizáveis
- README completo, explicando propósito, arquitetura e como executar o projeto
- Dockerfile para garantir reprodutibilidade do ambiente
- Logging estruturado em todas as etapas críticas
- Testes automatizados cobrindo as principais funções
- Segurança: .env, .gitignore, queries parametrizadas e IAM configurado corretamente

Conclusão

Construir pipelines de dados seguros não é um diferencial opcional — é o padrão mínimo esperado de qualquer profissional que deseja atuar com Engenharia de Dados de forma séria. As práticas apresentadas aqui (variáveis de ambiente, prevenção de SQL Injection, logging, tratamento de erros, IAM e gerenciamento de secrets) formam a base sobre a qual qualquer pipeline robusto deve ser construído, independentemente da escala do projeto.

Aliar esse conhecimento técnico ao uso produtivo de ferramentas de IA generativa é hoje uma vantagem competitiva real: profissionais que sabem usar IA para acelerar entrega, sem abrir mão de qualidade e segurança, se destacam em processos seletivos. Este ebook é também um exercício de comunicação técnica — mostrar que é possível explicar conceitos complexos de forma clara é tão importante quanto saber implementá-los.

Links

- GitHub: github.com/alexandrevisgueira
- LinkedIn: [linkedin.com/in/alexandre-ferreira-094382384](https://www.linkedin.com/in/alexandre-ferreira-094382384)

Ebook produzido como parte do desafio DIO utilizando IA Generativa — Alexandre, 2026